MAAT v1.6 Guided SAT Search

Structural Search Geometry for Transparent DPLL Branching

Christof Krieg
MAAT Project – Independent Research
https://maat-research.com

2026

Abstract

Previous MAAT SAT papers studied hardness prediction: graph-local defects, multi-scale frustration fields, CDCL statistics, and projection decompositions. Those papers asked whether structural diagnostics predict how hard a SAT instance is. This paper asks a different question: *can MAAT v1.6 structural search geometry actively steer the SAT search itself?*

We implement a transparent DPLL solver with unit propagation and compare branching rules on deterministically generated SAT-family instances: random 3-SAT, planted 3-SAT, modular 3-SAT, 3-XOR CNF, graph coloring, and pigeonhole formulas. Baselines include first-unassigned, random, degree, Jeroslow-Wang, and MOMS. The MAAT brancher scores variables using verification gain, short-clause pressure, local connectivity, polarity balance, and the v1.2.1 robustness closure.

The result is informative but not triumphant. The MAAT v1.6 brancher solves all tested instances and improves over degree, first-unassigned, and random branching in aggregate. However, classical SAT-specific heuristics remain stronger: MOMS has the lowest median cost, followed by Jeroslow-Wang, while MAAT v1.6 ranks third. Thus the first active SAT-search test supports a modest conclusion: structural search geometry provides a usable active branching signal, but in its present form it does not yet compete with the strongest classical SAT-specific heuristics.

1 Purpose

MAAT v1.6 reframes structural selection as search geometry. In the SAT setting, the search space consists of partial assignments

$$X_0 \to X_1 \to X_2 \to \cdots ,$$

and the generator is the solver transition rule: choose a variable, assign a polarity, propagate implications, and backtrack when a conflict occurs.

Earlier SAT papers in this series were passive. They measured or predicted hardness using DPLL node counts, CDCL statistics, or defect-field projections. The present paper is active. It asks whether structural coordinates can influence which variable is chosen next.

The scope is deliberately narrow. This is not a proof of $P \neq NP$, not a CDCL solver competition, not an industrial SAT benchmark, and not a replacement for modern solvers such as Glucose, MiniSat, CaDiCaL, or Kissat. It is a controlled first test of the v1.6 idea:

> *structural supports should modulate search decisions, not merely describe the difficulty of a completed run.*

2 SAT as Structural Search Geometry

Let X denote a partial assignment. Let $\mathcal{F}$ be a CNF formula. At each solver state we consider the active unsatisfied clauses and compute candidate-level quantities for each unassigned variable v.

The generic v1.6 search functional is

$$F_{\text{search}}(v;X) = \lambda_D d_{\text{ver}}(v;X) + \lambda_E E(v;X) + \lambda_M M_{\text{struct}}(v;X) + \lambda_R(1 - R_{\text{rob}}(v;X)) - \lambda_V V(v;X). \tag{1}$$

For branching, lower cost corresponds to higher priority. In the implementation below it is more convenient to use an equivalent priority score, where larger is better.

The operational supports are:

- **Verification gain:** the larger of positive/negative occurrence counts in currently active clauses.
- **Energy pressure:** a Jeroslow-Wang-style short-clause weight.
- H: sign-consistency support, penalising extreme polarity imbalance.
- B: polarity balance support.
- S: short-clause propagation pressure.
- V: variable-neighbour connectivity in the active clause-variable graph.
- R_{rob}: v1.2.1 robustness closure,

$$R_{\text{resp}} = (HBV)^{1/3}, \qquad R_{\text{rob}} = \min\Big(R_{\text{resp}}, (HBSV)^{1/4}\Big).$$

The active MAAT priority is:

$$\boxed{\begin{aligned} P_{\text{MAAT}}(v;X) = {} & 0.44\, G_{\text{ver}}(v) + 0.24\, E_{\text{JW}}(v) + 0.14\, S(v) + 0.13\, V(v) + 0.08\, R_{\text{rob}}(v) \\ & - 0.035 \left[- \sum_{a \in \{H,B,S,V,R_{\text{rob}}\}} \log(\varepsilon + \Gamma_a(v)) \right]. \end{aligned}} \tag{2}$$

A heavier structural variant is also tested. It gives more weight to S, V, R_{rob} and less to direct verification gain. This tests whether structural support can overrule the target signal.

3 Benchmark Design

The benchmark uses a transparent DPLL solver with unit propagation. There is no clause learning, no restarts, and no VSIDS. This is intentional: the goal is to isolate branching-rule behaviour before embedding MAAT into a full CDCL architecture.

The benchmark is deliberately light-to-moderate rather than adversarially hard. Most instances are solved by almost every reasonable heuristic. Consequently, the experiment primarily measures branching-cost ordering on solvable generated instances, not separation by hard timeouts or industrial CDCL performance.

The solver is tested on 141 generated instances from six SAT families:

Family	Role
random 3-SAT	phase-sensitive random formulas.
planted 3-SAT	satisfiable formulas with hidden assignment structure.
modular 3-SAT	clustered variable-community formulas.
3-XOR CNF	parity-like constraints converted to CNF.
graph coloring	structured graph constraint formulas.
pigeonhole	small unsatisfiable combinatorial formulas.

The branching rules are:

- first unassigned variable,
- random unassigned variable,
- degree heuristic,
- Jeroslow-Wang,
- MOMS,
- MAAT v1.6 branch,
- MAAT v1.6 heavy.

The fixed decision budget is 5500. The main cost measure is

$$\log(1 + D + 0.35C + 0.10P + 0.20B)\,, \tag{3}$$

where D is decisions, C conflicts, P unit propagations, and B backtracks.

4 Graphical SAT Representation

SAT formulas can be visualised as graphs in a way analogous to dense mathematical graph drawings. The picture is not a geometric embedding of a theorem, but a diagnostic view of the search space. Two graph projections are useful:

- the **variable co-occurrence graph**, where variables are connected when they appear in a common clause;
- the **clause-variable bipartite graph**, where variables and clauses are separate node types and signed edges indicate positive or negative literals.

In the figures below, variable colour is the MAAT v1.6 branching priority at the root state, and node size tracks the connectivity support V. Thus high-priority variables become visible as structural pressure points in the SAT instance. The figures should be read as root-state feature maps: they show what the structural brancher attends to before search begins. Their role is not to explain performance by themselves, but to connect the numerical brancher to visible pressure points in the formula.

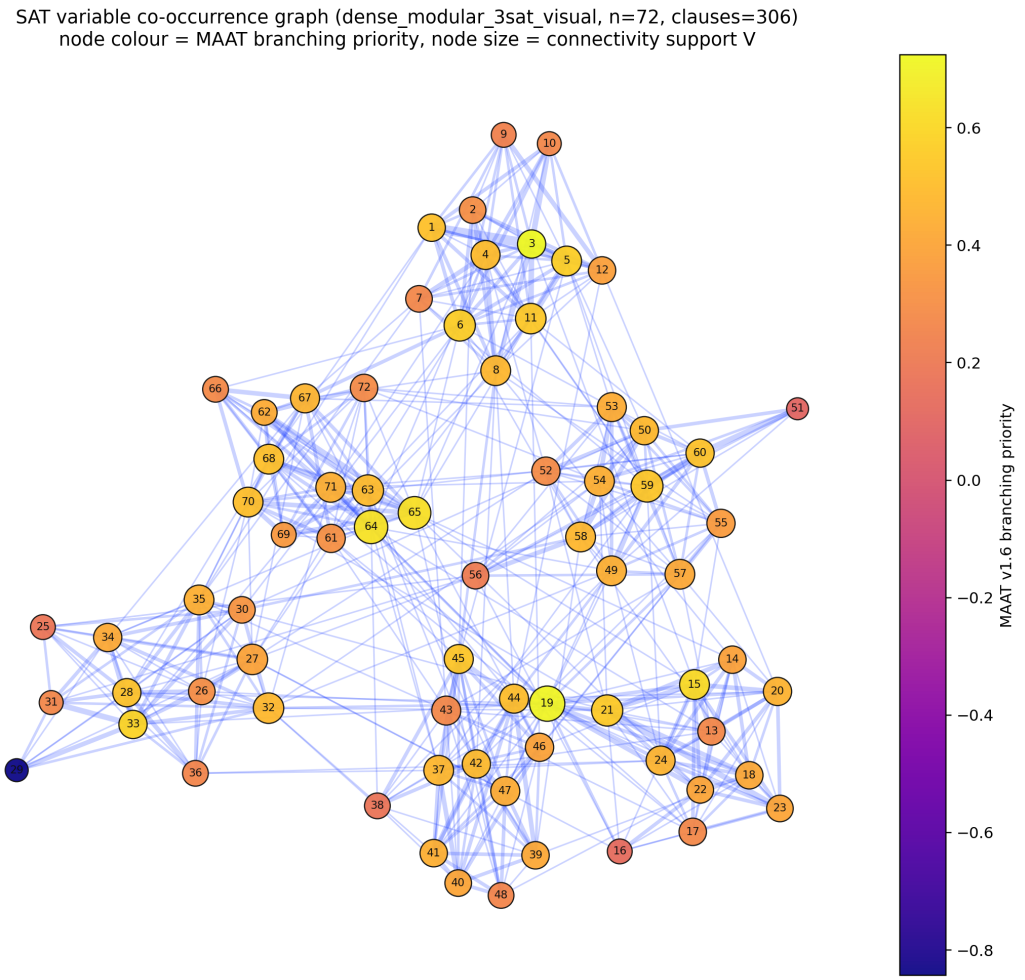

Figure 1: Variable co-occurrence graph for a dense modular 3-SAT display instance. Edges indicate variables appearing together in a clause; colour shows MAAT v1.6 branching priority and node size shows connectivity support.

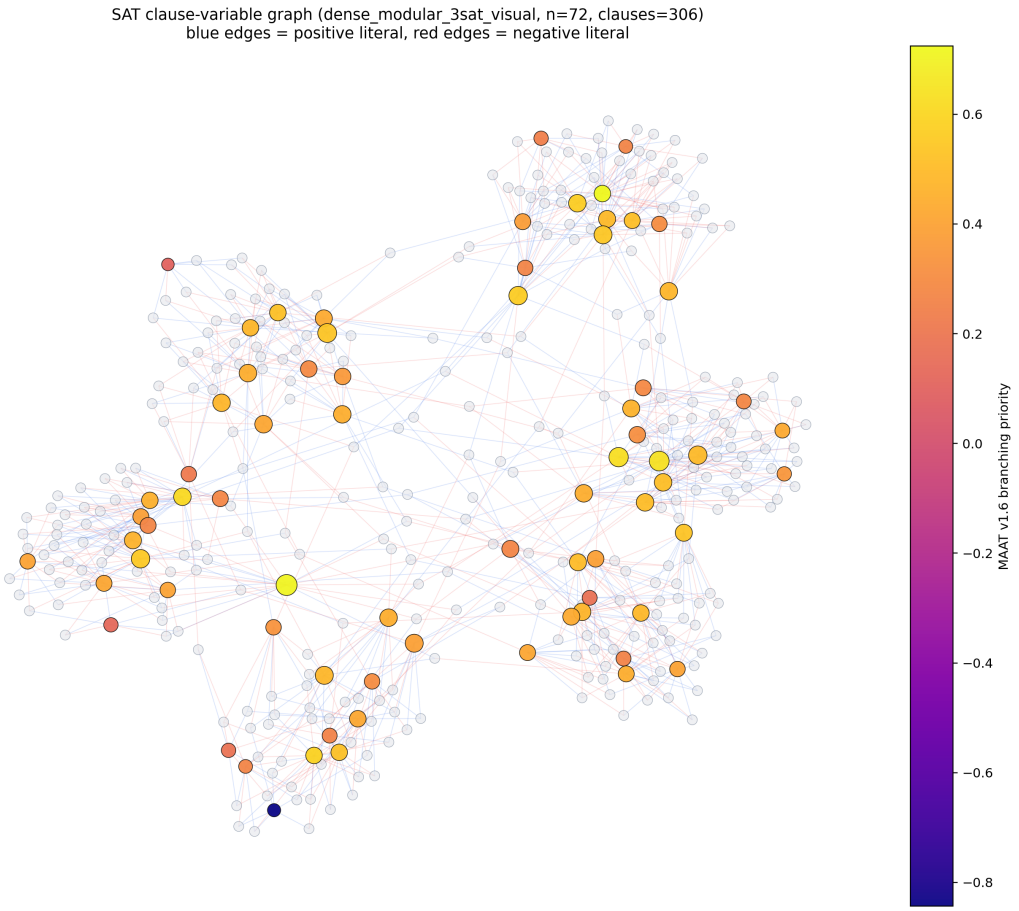

Figure 2: Clause-variable bipartite graph for the same display instance. Pale nodes are clauses; coloured nodes are variables. Blue edges denote positive literals and red edges denote negative literals.

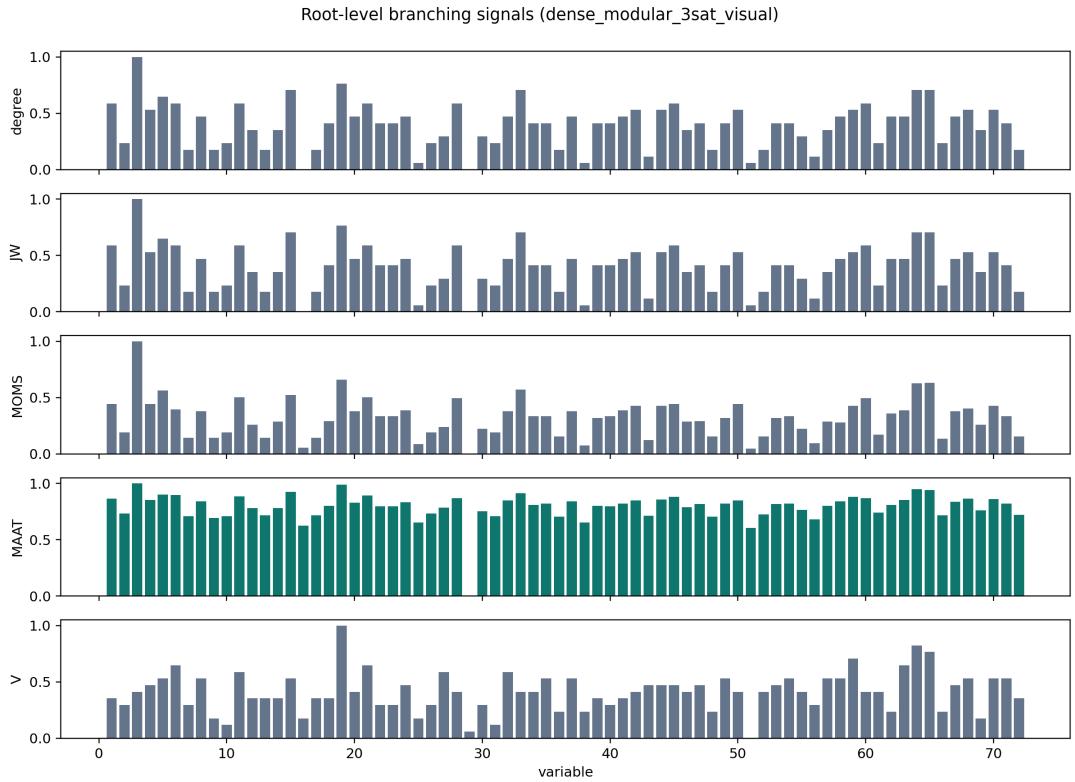

Figure 3: Branching-score components for the same display instance. The plot links the graph visualisations to the actual MAAT priority signal: high-scoring variables combine verification gain, short-clause pressure, connectivity, and robustness closure.

5 Results

Table 1: Aggregate branching results. Lower cost is better.

Heuristic	**Solved**	**Median decisions**	**Median conflicts**	**Median cost**
MOMS	1.000	9.0	2.0	2.6283
Jeroslow-Wang	1.000	11.0	2.0	2.7788
MAAT v1.6 branch	1.000	12.0	2.0	2.9178
MAAT v1.6 heavy	1.000	12.0	2.0	2.9232
Degree	1.000	14.0	3.0	3.0587
First	0.993	17.0	6.0	3.2696
Random	1.000	18.0	5.0	3.3087

The improvement over degree, first-unassigned, and random branching is a moderate but real positive signal. It shows that the structural coordinates are usable for active branching. It should not be read as evidence of competitive SAT performance, since MOMS and Jeroslow-Wang remain better on the same benchmark.

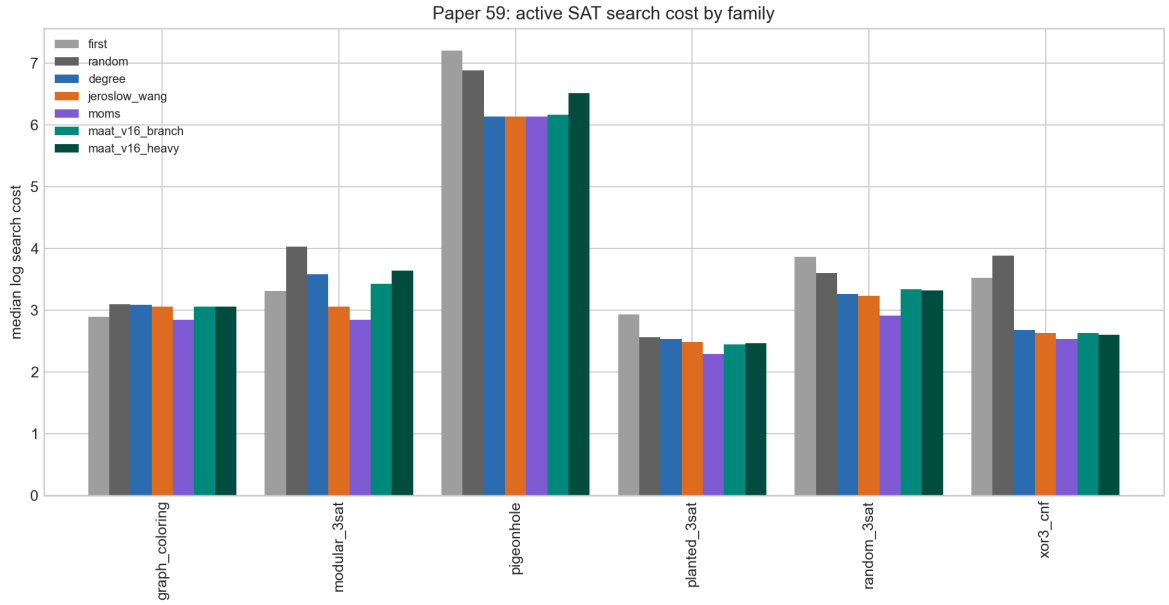

Figure 4: Median search cost by SAT family and branching heuristic.

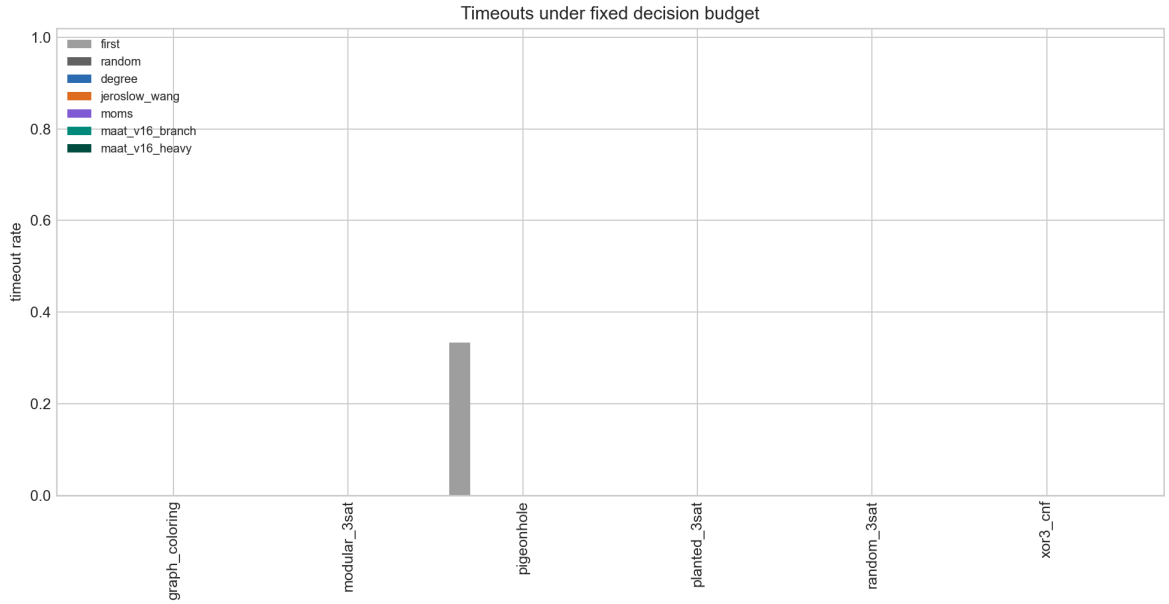

Figure 5: Timeout rate under the fixed decision budget. The benchmark is small enough that most heuristics solve all instances; first unassigned times out on one pigeonhole case.

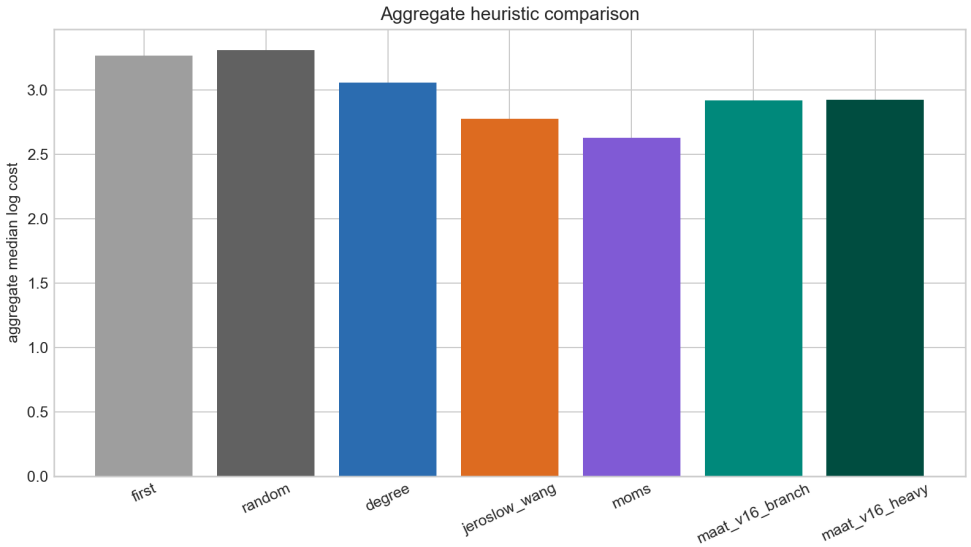

Figure 6: Aggregate median log search cost. MAAT v1.6 improves over degree, first, and random branching, but not over MOMS or Jeroslow-Wang.

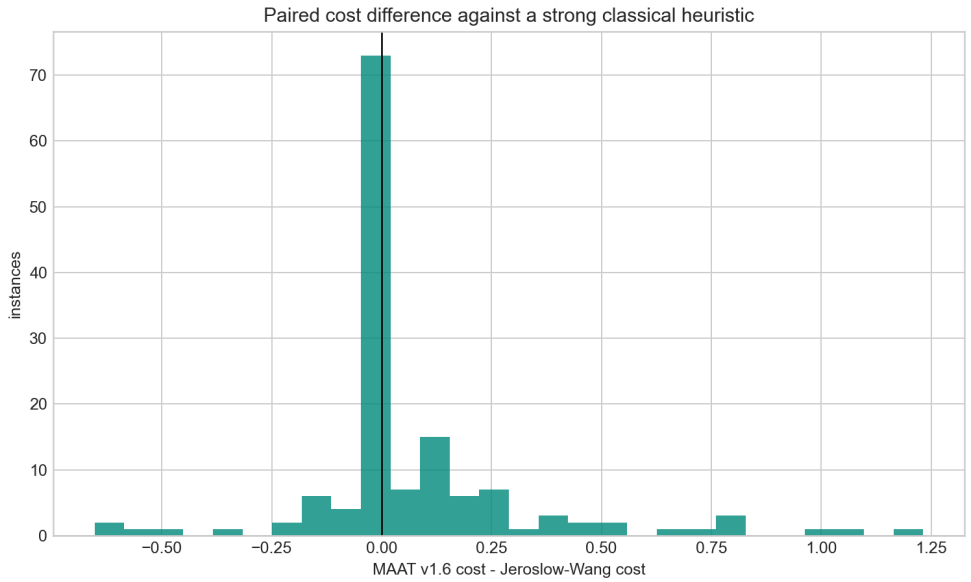

Figure 7: Paired cost difference against Jeroslow-Wang. Negative values favour MAAT; positive values favour Jeroslow-Wang. Most instances do not favour MAAT against this strong classical baseline.

The paired comparisons are:

Baseline	**Median cost difference**	**Fraction MAAT better**
Degree	−0.0091	0.5248
Jeroslow-Wang	0.0000	0.1631
MOMS	0.1344	0.1986

6 Interpretation

The benchmark gives a clear but modest result. The active MAAT brancher is a functional branching heuristic. It is not merely a passive hardness score. It solves all tested instances and improves over random, first-unassigned, and degree branching in aggregate. This is the positive signal of the paper: MAAT v1.6 provides an active structural branching signal that is usable in a solver loop, although the effect is moderate on this relatively easy benchmark.

However, the main negative result is equally important. MAAT v1.6 does not beat MOMS or Jeroslow-Wang in this transparent DPLL setting. This means that the current structural priority score does not yet encode the domain-specific strength of established SAT branching heuristics. The graph visualisations help show what the MAAT priority is seeing: structurally central, connected variables with local support pressure, plus the component-level mixture shown in Figure 3. The performance tables show the limitation of that view: MOMS and Jeroslow-Wang exploit short-clause pressure more directly than the current structural score.

The heavy MAAT variant is not better than the anchored variant. In fact, it is slightly worse in aggregate (2.9232 versus 2.9178 median log cost). This repeats the lesson of the v1.6 grid-search benchmark: structural supports are useful only when anchored to the immediate verification task. If the structural mode becomes too dominant, it does not automatically improve search. Thus the heavy variant functions as an important negative control, not as a failed tuning attempt to hide.

The positive interpretation is therefore narrow:

> *MAAT v1.6 provides a usable structural branching signal, but in its present form it does not yet compete with the strongest SAT-specific heuristics.*

7 Relation to Previous SAT Papers

Paper 50 showed that SAT hardness is graph-local and phase-aware. Paper 50b showed that aggressive scalar compression loses frustration signal. Paper 55 tested defect-field features against CDCL hardness statistics from standard SAT families. Paper 57 decomposed that signal into mean, tail, cluster, and scale projections.

Paper 59 closes a different loop. It uses the same broad lesson, but changes the role of MAAT:

from predicting solver effort to steering solver decisions.

This is the first active SAT-search benchmark in the MAAT series.

8 Limitations

- The solver is DPLL with unit propagation, not full CDCL.
- There is no clause learning, restart policy, VSIDS, LRB, CHB, or phase saving.
- The benchmark is small and generated synthetically.
- Most instances are solved by all nontrivial heuristics, so the benchmark measures cost differences on light-to-moderate instances rather than hard timeout separation.
- Because the instances are mostly easy for reasonable heuristics, the results should be read as evidence of a branching signal, not as a strong SAT hardness or scalability claim.
- The MAAT priority weights are fixed heuristic weights, not learned from a training set.
- A serious next test should integrate MAAT features as a secondary signal inside a CDCL solver or compare against solver traces from PySAT, CaDiCaL, or Kissat.

9 Reproducibility

The experiment is stored in:

`experiments/sat_v16_guided_search_paper59/`

Run:

```
cd experiments/sat_v16_guided_search_paper59
python3 sat_v16_guided_search_paper59.py
```

Outputs are written to:

`outputs_paper59/`

The public repository is:

https://github.com/Chris4081/structural-selection-principle

No external CNF dataset is redistributed. All formulas are generated locally by the script from standard synthetic SAT families. CSV, JSON, and PNG files are derived reproducibility artifacts.

10 Conclusion

Paper 59 tests MAAT v1.6 where it matters for SAT: not as a descriptive hardness score, but as an active branching rule.

The result is useful because it is bounded. MAAT v1.6 branching is viable and beats degree, first-unassigned, and random baselines, but it does not surpass MOMS or Jeroslow-Wang. The heavy structural variant does not improve the anchored version, so simply increasing structural weight is not enough. This supports further work on structural SAT search only if the claim remains modest:

MAAT may be useful as an auxiliary structural signal, not yet as a standalone SAT heuristic.

The next step is a CDCL-level experiment in which MAAT defect fields modulate an established branching signal rather than replacing it.

References

[1] M. Davis, G. Logemann, and D. Loveland, "A Machine Program for Theorem-Proving," *Communications of the ACM* **5**, 394–397 (1962).

[2] R. G. Jeroslow and J. Wang, "Solving Propositional Satisfiability Problems," *Annals of Mathematics and Artificial Intelligence* **1**, 167–187 (1990).

[3] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability," *IEEE Transactions on Computers* **48**, 506–521 (1999).

[4] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, "Chaff: Engineering an Efficient SAT Solver," Proceedings of DAC (2001).

[5] N. Eén and N. Sörensson, "An Extensible SAT-solver," Proceedings of SAT (2003).

[6] G. Audemard and L. Simon, "Predicting Learnt Clauses Quality in Modern SAT Solvers," Proceedings of IJCAI (2009).

[7] C. Krieg, "CDCL Hardness Prediction on Standard SAT Families," MAAT Project preprint (2026).

[8] C. Krieg, "Structural Search Geometry," MAAT Project preprint (2026).